

Functional Test Cases

Library Management System

Test Design Team (Generated by Gemini)

June 13, 2025

Note: These test cases are designed from a black-box, user-centric perspective, based on existing unit test files. Actual results and status are to be filled during test execution.

0.1 Server-Side Functional Tests

0.1.1 Authentication & Authorization

Table 1: Authentication and Authorization Test Cases

Test Case ID	Test Title	Test Steps	Expected Results
Sub-Section: User Registration			
TC_AUTH_REG.001	Successful new user registration	• Send a request to the registration endpoint. • Provide a valid and unique email, a password, and a name.	• System accepts the request. • A new user record is added to the database. • The API returns the user details with a status of 201.
TC_AUTH_REG.002	Attempt to register with an existing email	• Send a request to the registration endpoint with an email that already exists.	• System rejects the request. • An error message "Email already exists" is returned. • The API returns a status of 400.
TC_AUTH_REG.003	Handle server error during registration	• Trigger a server-side error during the user creation process (e.g., database connection fails).	• System handles the error gracefully. • A generic error message is returned. • The API returns a status of 500.
Sub-Section: User Login			
TC_AUTH_LOGIN.001	Successful login with valid credentials	• Send a request to the login endpoint with a valid email and correct password.	• User is authenticated successfully. • A JWT token is returned in the response. • The API returns a status of 200.
TC_AUTH_LOGIN.002	Attempt login with invalid password	• Send a request to the login endpoint with a valid email but an incorrect password.	• Login fails. • An error message "Invalid password" is returned. • The API returns a status of 401.
TC_AUTH_LOGIN.003	Attempt login with non-existent email	• Send a request to the login endpoint with an email that does not exist in the database.	• Login fails. • An error message "Credentials not found" is returned. • The API returns a status of 401.
TC_AUTH_LOGIN.004	Handle server error during login	• Trigger a server-side error during the login process (e.g., database lookup fails).	• System handles the error gracefully. • A generic error message is returned. • The API returns a status of 500.

0.1.2 Author Management

Table 2: Author Management Test Cases

Test Case ID	Test Title	Test Steps	Expected Results
Sub-Section: Author CRUD Operations			
TC_AUTHOR.CREATE_001	Create a new author successfully	• Send a POST request with a valid first and last name.	• A new author is created. • The new author's ID is returned. • API returns status 201.
TC_AUTHOR.READ_001	Retrieve all authors	• Send a GET request to the all authors endpoint.	• A list of all authors is returned. • API returns status 200.
TC_AUTHOR.READ_002	Retrieve a single author by valid ID	• Send a GET request with a valid author ID.	• The corresponding author is returned. • API returns status 200.
TC_AUTHOR.READ_003	Attempt to retrieve a non-existent author	• Send a GET request with an invalid author ID.	• Request fails. • An error message is returned. • API returns status 404.
TC_AUTHOR.UPDATE_001	Update an existing author successfully	• Send a PUT request with a valid author ID and updated data.	• The author's record is updated. • The updated author is returned. • API returns status 200.
TC_AUTHOR.DELETE_001	Delete an existing author successfully	• Send a DELETE request with a valid author ID.	• The author's record is deleted. • A success message is returned. • API returns status 204.

0.1.3 Book Management

Table 3: Book Management Test Cases

Test Case ID	Test Title	Test Steps	Expected Results
Sub-Section: Book CRUD Operations			
TC_BOOK.CREATE_001	Create a new book successfully	• Send a POST request with all required, valid book details.	• A new book record is created. • The new book data is returned. • API returns status 201.
TC_BOOK.READ_001	Retrieve all books	• Send a GET request to the all books endpoint.	• A list of all book records is returned. • API returns status 200.
TC_BOOK.UPDATE_001	Update an existing book successfully	• Send a PUT request with a valid book ID and updated data.	• The book's record is updated. • The updated book data is returned. • API returns status 200.
TC_BOOK.DELETE_001	Delete an existing book successfully	• Send a DELETE request with a valid book ID.	• The book's record is deleted. • A success message is returned. • API returns status 200.

0.1.4 Borrowal Management

Table 4: Borrowal Management Test Cases

Test Case ID	Test Title	Test Steps	Expected Results
Sub-Section: Borrowal Operations			
TC_BORROW_CREATE.001	Create new borrowal for available book	• Send a POST request with a valid book ID (for an available book) and member ID.	• A new borrowal record is created. • The book's status is updated to 'borrowed'. • API returns status 201.
TC_BORROW_CREATE.002	Attempt borrowal for unavailable book	• Send a POST request for a book that is already borrowed.	• Request is rejected with status 400. • An error message "Book is not available" is returned. • API returns status 400.
TC_BORROW_UPDATE.001	Update borrowal record (return book)	• Send a PUT request with a valid borrowal ID and a return date.	• The borrowal record is updated. • The associated book's status is updated to 'available'. • API returns status 200.

0.2 Client-Side Functional Tests

0.2.1 Login Form Component

Table 5: Login Form Test Cases

Test Case ID	Test Title	Test Steps	Expected Results
Sub-Section: Form Display and Submission			
TC_LOGINFORM_UI_001	Verify login form renders correctly	Navigate to the login page.	- The login form is visible. - Email and password fields are visible. - A 'Login' button is visible.
TC_LOGINFORM_SUBMIT_001	Successful login submission	- Enter a valid email and password into the form fields. - Click the 'Login' button.	- A login request is sent to the backend. - The user is redirected to the dashboard upon successful login.
Sub-Section: Form Validation			
TC_LOGINFORM_VAL_001	Attempt submission with invalid email format	- Enter an invalidly formatted email (e.g., "test@test"). - Attempt submission.	- Form submission fails. - An error message "Invalid email address" is displayed near the email field.
TC_LOGINFORM_VAL_002	Attempt submission with empty password	- Enter a valid email. - Leave the password field empty. - Attempt to submit the form.	- Form submission fails. - An error message "Password is required" is displayed near the password field.

0.2.2 Data Formatting Utilities

Table 6: Data Formatting Utility Test Cases

Test Case ID	Test Title	Test Steps	Expected Results
Sub-Section: Number Formatting			
TC_FORMAT_NUM_001	Format number as currency	Call the 'fCurrency' function with a number (e.g., 12345).	The number is returned as formatted currency (e.g., "\$12,345.00").
TC_FORMAT_NUM_002	Shorten a large number	Call the 'fShortenNumber' function with a large number (e.g., 1234567).	The number is returned in compact, human-readable format (e.g., "1.23m").
TC_FORMAT_NUM_003	Format number as data size	Call the 'fData' function with a number representing bytes (e.g., 1234567).	The number is converted to a human-readable data size (e.g., "1.18 MB").
Sub-Section: Time Formatting			
TC_FORMAT_TIME_001	Format a date string	Call the 'fDate' function with a valid date object or string.	The date is returned in "dd/MM/yyyy" format.
TC_FORMAT_TIME_002	Format date as relative time	Call the 'fToNow' function with a date from the past.	A human-readable relative time string is returned (e.g., "2 hours ago").